

File: aperture_macro.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/testes_gerber

```
from __future__ import annotations

import sys

from PyQt6.QtWidgets import QApplication

from gerber_viewer.gui.mainwindow import GerberMacroViewer


def main():
    app = QApplication(sys.argv)
    window = GerberMacroViewer()
    window.resize(900, 700)
    window.show()
    sys.exit(app.exec())


if __name__ == "__main__":
    main()
```

File: apertures.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/testes_gerber/gerber_viewer

```
from __future__ import annotations

import math
from typing import Dict, List

from .config import GerberConfig
from .constants import INCH_TO_MM


class ApertureMacro:
    def __init__(self, name: str):
        self.name = name
        # cada item = (code, expo, params)
        self.primitives: List[tuple[int, int, List[str]]] = []

    def add_primitive(self, code: int, expo: int, params: List[str]):
        self.primitives.append((code, expo, params))

    def render(
        self,
        scale_x: float = 1.0,
        scale_y: float = 1.0,
        rot_deg: float = 0.0,
```

```

trans: tuple[float, float] = (0.0, 0.0),
) -> List[List[tuple[float, float]]]:
    """

```

Retorna lista de polígonos (listas de pontos) já transformados.
 O parâmetro `trans` permite posicionar a macro em coordenadas do mundo (útil para flashes de stencils).

```

    """

```

```

polys: List[List[tuple[float, float]]] = []
theta = math.radians(rot_deg)
cos_t, sin_t = math.cos(theta), math.sin(theta)

```

```

def transform(pt: tuple[float, float]) -> tuple[float, float]:

```

```

    x = pt[0] * scale_x
    y = pt[1] * scale_y
    # rotaciona
    xr = x * cos_t - y * sin_t
    yr = x * sin_t + y * cos_t
    # translada
    return (xr + trans[0], yr + trans[1])

```

```

for code, expo, p in self.primitives:

```

```

    if code == 4: # outline
        n = int(p[0])
        coords: List[tuple[float, float]] = []
        for i in range(n):
            x = float(p[1 + 2 * i])
            y = float(p[1 + 2 * i + 1])
            coords.append(transform((x, y)))
        if expo == 1 and coords:
            polys.append(coords + [coords[0]]) # fecha explicitamente
    return polys

```

```

def parse_macro(lines: List[str]) -> ApertureMacro:

```

```

    """Recebe lista de linhas entre %AM...* e % e devolve ApertureMacro."""

```

```

    header = lines[0] # ex: "%AMacap0165_180*"

```

```

    name = header[3:-1]

```

```

    mac = ApertureMacro(name)

```

```

    # junta só as linhas que NÃO são o "%" final

```

```

    body = "".join(l for l in lines[1:] if not l.strip() == '%')

```

```

    # só tokens que começam com dígito (4,1,42,...) e não "%", ou vazios

```

```

    prims = [
        s.strip()
        for s in body.split('*')
        if s.strip() and s.strip()[0].isdigit()
    ]

```

```

    for prim in prims:

```

```

        tokens = prim.split(',')

```

```

        code = int(tokens[0])

```

```

        expo = int(tokens[1])

```

```

        params = tokens[2:]

```

```
    mac.add_primitive(code, expo, params)
return mac
```

```
def parse_all_macros(gerber_lines: List[str]) -> Dict[str, ApertureMacro]:
```

```
    """
```

Varre o arquivo Gerber linha a linha e extrai todos os blocos de macro:

```
    %AMnome*
```

```
    ...
```

```
    %
```

ou o formato clássico:

```
    %AMnome*
```

```
    ...
```

```
    ...0.00000*0%
```

```
    """
```

```
    macros: Dict[str, ApertureMacro] = {}
```

```
    i = 0
```

```
    n = len(gerber_lines)
```

```
    while i < n:
```

```
        line = gerber_lines[i].strip()
```

```
        if line.startswith("%AM"):
```

```
            block = [line]
```

```
            i += 1
```

```
            while (
```

```
                i < n
```

```
                and gerber_lines[i].strip() != "%"
```

```
                and not gerber_lines[i].strip().endswith("*%")
```

```
            ):
```

```
                block.append(gerber_lines[i].rstrip("\n"))
```

```
                i += 1
```

```
            if i < n:
```

```
                block.append(gerber_lines[i].rstrip("\n"))
```

```
            macro = parse_macro(block)
```

```
            macros[macro.name] = macro
```

```
            i += 1
```

```
    return macros
```

```
class ApertureInstance:
```

```
    """
```

Representa uma abertura associada a um D-code, após o parsing de %ADD...%:

- kind:

"circle" → círculo simples (C,diam)

"rect" → retângulo (R,WxH)

"oval" → oval/obround (O,WxH)

"macro" → macro Gerber (%AMxxx*)

- params: dicionário com parâmetros em MILÍMETROS (para shapes simples) ou dados da macro (nome, rotação, escala, ...).

"""

```
def __init__(self, kind: str, **params):
    self.kind = kind
    self.params = params
```

```
def _text_to_mm(val: str, cfg: GerberConfig) -> float:
    """Converte um valor numérico do ADD (diâmetros, larguras, etc.) para mm."""
    v = float(val)
    if cfg.unit == "inch":
        return v * INCH_TO_MM
    return v
```

```
def parse_add(
    gerber_lines: List[str],
    cfg: GerberConfig,
) -> Dict[int, ApertureInstance]:
    """
```

Analisa todas as declarações %ADD...% e constrói um mapa:

dcode (int) -> ApertureInstance

"""

```
apertures: Dict[int, ApertureInstance] = {}
```

```
for raw in gerber_lines:
```

```
    line = raw.strip()
```

```
    if not line.startswith("%ADD"):
```

```
        continue
```

```
    # remove delimitadores iniciais/finais
```

```
    s = line.strip("%").rstrip("*")
```

```
    # agora s é algo como "ADD10C,0.00100" ou "ADD102acap0150_90"
```

```
    if not s.startswith("ADD"):
```

```
        continue
```

```
    s = s[3:] # remove "ADD"
```

```
    # dcode = sequência inicial de dígitos
```

```
    i = 0
```

```
    while i < len(s) and s[i].isdigit():
```

```
        i += 1
```

```
    if i == 0:
```

```
        continue
```

```
    dcode = int(s[:i])
```

```
    rest = s[i:] # restante: tipo + parâmetros ou nome de macro
```

```
    if not rest:
```

continue

```
# Shape simples: C, R ou O
if rest[0] in ("C", "R", "O"):
    shape_type = rest[0]
    params_str = rest[1:] # ex: ",0.02000" ou ",0.02000X0.01400"
    if params_str.startswith(","):
        params_str = params_str[1:]

    if "X" in params_str:
        a_str, b_str = params_str.split("X", 1)
    else:
        a_str, b_str = params_str, None

    if shape_type == "C":
        dia_mm = _text_to_mm(a_str, cfg)
        apertures[dcode] = ApertureInstance("circle", dia_mm=dia_mm)
    elif shape_type == "R":
        if b_str is None:
            continue
        w_mm = _text_to_mm(a_str, cfg)
        h_mm = _text_to_mm(b_str, cfg)
        apertures[dcode] = ApertureInstance(
            "rect",
            width_mm=w_mm,
            height_mm=h_mm,
        )
    elif shape_type == "O":
        if b_str is None:
            continue
        w_mm = _text_to_mm(a_str, cfg)
        h_mm = _text_to_mm(b_str, cfg)
        apertures[dcode] = ApertureInstance(
            "oval",
            width_mm=w_mm,
            height_mm=h_mm,
        )
    else:
        # Macro: o restante é o nome da macro (ex.: "acap0150_90")
        macro_name = rest
        apertures[dcode] = ApertureInstance("macro", macro_name=macro_name)

return apertures
```

File: config.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/testes_gerber/gerber_viewer

from __future__ import annotations

```
from typing import List
```

```
from .constants import INCH_TO_MM
```

```
class GerberConfig:
```

```
    """
```

```
    Configuração básica do arquivo Gerber:
```

```
    - unidade (inch/mm)
```

```
    - formato de coordenadas (número de dígitos inteiros e decimais)
```

```
    Ex.: %FSLAX26Y26*% → int=2, dec=6, unidade = polegadas (MOIN).
```

```
    """
```

```
    def __init__(self, unit: str = "inch", fmt_int: int = 2, fmt_dec: int = 6):
```

```
        self.unit = unit      # "inch" ou "mm"
```

```
        self.fmt_int = fmt_int # dígitos inteiros
```

```
        self.fmt_dec = fmt_dec # dígitos decimais
```

```
def parse_gerber_config(gerber_lines: List[str]) -> GerberConfig:
```

```
    """
```

```
    Lê o cabeçalho do Gerber e retorna unidade e formato de coordenadas.
```

```
    Procura por:
```

```
    - %FS...* → formato (ex.: %FSLAX26Y26*%)
```

```
    - %MO...* → unidade (MOIN/MOMM)
```

```
    """
```

```
    cfg = GerberConfig()
```

```
    for line in gerber_lines:
```

```
        s = line.strip()
```

```
        if s.startswith("%FS"):
```

```
            # Exemplo: %FSLAX26Y26*%
```

```
            content = s.strip("%*") # "FSLAX26Y26"
```

```
            if "X" in content and "Y" in content:
```

```
                ix = content.index("X")
```

```
                iy = content.index("Y")
```

```
                xfmt = content[ix + 1:iy] # "26"
```

```
                if len(xfmt) >= 2 and xfmt[0].isdigit() and xfmt[1].isdigit():
```

```
                    cfg.fmt_int = int(xfmt[0])
```

```
                    cfg.fmt_dec = int(xfmt[1])
```

```
            elif s.startswith("%MO"):
```

```
                up = s.upper()
```

```
                if "MOIN" in up:
```

```
                    cfg.unit = "inch"
```

```
                elif "MOMM" in up:
```

```
                    cfg.unit = "mm"
```

```
    return cfg
```

```
def parse_coord(val_str: str, cfg: GerberConfig) -> float:
```

```
    """
```

Converte uma string de coordenada Gerber (sem ponto) em valor numérico na unidade base do arquivo (inch ou mm), considerando formato FS (número de dígitos inteiros e decimais) e zero suppression 'L' (leading).

```
"""
```

```
s = val_str.strip()
```

```
if not s:
```

```
    return 0.0
```

```
sign = -1 if s[0] == "-" else 1
```

```
if s[0] in "+-":
```

```
    s = s[1:]
```

```
total_len = cfg.fmt_int + cfg.fmt_dec
```

```
# leading zero suppression → faz left-pad até o tamanho esperado
```

```
s = s.rjust(total_len, "0")
```

```
int_part = int(s[:cfg.fmt_int])
```

```
dec_part = int(s[cfg.fmt_int:]) / (10 ** cfg.fmt_dec)
```

```
return sign * (int_part + dec_part)
```

```
def to_mm_from_unit(v: float, cfg: GerberConfig) -> float:
```

```
    """Converte um valor na unidade do arquivo (inch/mm) para mm."""
```

```
    if cfg.unit == "inch":
```

```
        return v * INCH_TO_MM
```

```
    return v
```

File: constants.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/testes_gerber/gerber_viewer

```
INCH_TO_MM = 25.4
```

File: geometry.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/testes_gerber/gerber_viewer

```
from __future__ import annotations
```

```
import math
```

```
from typing import List
```

```
def circle_to_polys_mm(
```

```
    xc: float,
```

```
    yc: float,
```

```
    dia_mm: float,
```

```
    n_sides: int = 64,
```

```
) -> List[List[tuple[float, float]]]:
```

```
    """Aproxima um círculo por um polígono regular de n_sides lados."""
```

```

r = dia_mm / 2.0
pts: List[tuple[float, float]] = []
for i in range(n_sides):
    ang = 2 * math.pi * i / n_sides
    pts.append((xc + r * math.cos(ang), yc + r * math.sin(ang)))
if pts:
    pts.append(pts[0])
return [pts]

```

```

def rect_to_polys_mm(
    xc: float,
    yc: float,
    w_mm: float,
    h_mm: float,
) -> List[List[tuple[float, float]]]:
    """Gera o polígono de um retângulo alinhado aos eixos, centrado em (xc, yc)."""
    hw = w_mm / 2.0
    hh = h_mm / 2.0
    pts = [
        (xc - hw, yc - hh),
        (xc + hw, yc - hh),
        (xc + hw, yc + hh),
        (xc - hw, yc + hh),
    ]
    pts.append(pts[0])
    return [pts]

```

```

def oval_to_polys_mm(
    xc: float,
    yc: float,
    w_mm: float,
    h_mm: float,
    n_sides: int = 64,
) -> List[List[tuple[float, float]]]:
    """
    Aproxima uma abertura OVAL (obround) por uma elipse de eixos w_mm x h_mm.
    Não é exatamente a construção "retângulo + semi-círculos", mas costuma
    ser suficiente para visualização / máscara raster.
    """
    a = w_mm / 2.0
    b = h_mm / 2.0
    pts: List[tuple[float, float]] = []
    for i in range(n_sides):
        ang = 2 * math.pi * i / n_sides
        pts.append((xc + a * math.cos(ang), yc + b * math.sin(ang)))
    if pts:
        pts.append(pts[0])
    return [pts]

```

File: parser.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/testes_gerber/gerber_viewer

```
from __future__ import annotations
```

```
import math
```

```
from dataclasses import dataclass
```

```
from typing import Dict, List, Tuple, Optional
```

```
from .apertures import ApertureInstance, ApertureMacro
```

```
from .config import GerberConfig, parse_coord, to_mm_from_unit
```

```
from .geometry import (
```

```
    circle_to_polys_mm,
```

```
    rect_to_polys_mm,
```

```
    oval_to_polys_mm,
```

```
)
```

```
Point = Tuple[float, float]
```

```
@dataclass
```

```
class GerberObject:
```

```
    """
```

```
    Representa um elemento geométrico já interpretado do Gerber, em mm:
```

```
    - kind:
```

```
        "flash_circle", "flash_rect", "flash_oval", "flash_macro", "region"
```

```
    - dcode:
```

```
        D-code ativo no momento do flash (ou None para regiões)
```

```
    - x_mm, y_mm:
```

```
        posição do flash (para regiões pode ser None ou centro aproximado)
```

```
    - params:
```

```
        dicionário com parâmetros geométricos (ex.: dia_mm, width_mm, etc.)
```

```
    - polygon_mm:
```

```
        lista de pontos (x,y) em mm, já fechada (primeiro == último).
```

```
    """
```

```
    id: int
```

```
    kind: str
```

```
    dcode: Optional[int]
```

```
    x_mm: Optional[float]
```

```
    y_mm: Optional[float]
```

```
    params: Dict[str, float | str]
```

```
    polygon_mm: List[Point]
```

```
def _parse_xy_from_line(line: str) -> tuple[str | None, str | None]:
```

```
    """Extrai substrings de X e Y de uma linha Gerber (sem interpretá-las)."""
```

```
    x_str = None
```

```
    y_str = None
```

```
    if "X" in line:
```

```

i = line.index("X") + 1
j = i
while j < len(line) and line[j] in "+-0123456789":
    j += 1
x_str = line[i:j]
if "Y" in line:
    i = line.index("Y") + 1
    j = i
    while j < len(line) and line[j] in "+-0123456789":
        j += 1
    y_str = line[i:j]
return x_str, y_str

```

```

def gerber_to_records(gerber_lines: List[str]) -> List[str]:

```

```

    """
    Converte a lista de linhas do arquivo Gerber em uma lista de
    "registros" separados por '*', como definido no padrão RS-274X.
    """

```

```

    text = ""
    for line in gerber_lines:
        s = line.strip()
        if not s:
            continue
        text += s

```

```

    parts = text.split("*")
    records: List[str] = []
    for p in parts:
        p = p.strip()
        if not p:
            continue
        records.append(p + "*")
    return records

```

```

def _get_draw_mode(record: str) -> str | None:

```

```

    """
    Retorna o modo de desenho (D01/D02/D03) de um registro, aceitando
    tanto D01/D02/D03 quanto D1/D2/D3.
    """

```

```

    if "D" not in record:
        return None
    idx = record.rfind("D")
    if idx == -1:
        return None
    j = idx + 1
    num = ""
    while j < len(record) and record[j].isdigit():
        num += record[j]
        j += 1

```

```
if not num:
    return None
```

```
if num in ("1", "01"):
    return "D01"
```

```
if num in ("2", "02"):
    return "D02"
```

```
if num in ("3", "03"):
    return "D03"
```

```
return None
```

```
def _parse_ij_from_record(record: str) -> tuple[str | None, str | None]:
    """Extraí substrings de I e J (centro relativo de arco) de um registro."""
    i_str = None
    j_str = None
    if "I" in record:
        k = record.index("I") + 1
        j = k
        while j < len(record) and record[j] in "+-0123456789":
            j += 1
        i_str = record[k:j]
    if "J" in record:
        k = record.index("J") + 1
        j = k
        while j < len(record) and record[j] in "+-0123456789":
            j += 1
        j_str = record[k:j]
    return i_str, j_str
```

```
def _get_g_code(record: str) -> int | None:
    """
    Extraí o código G (2, 3, 36, 37, etc.) de um registro.
    Retorna um inteiro (por ex. 2 para G02, 3 para G03) ou None.
    """
    s = record.lstrip()
    if "G" not in s:
        return None
    idx = s.index("G") + 1
    j = idx
    digits = ""
    while j < len(s) and s[j].isdigit():
        digits += s[j]
        j += 1
    if not digits:
        return None
    try:
        return int(digits)
    except ValueError:
        return None
```

```
def _approx_arc_points(
    x0: float,
    y0: float,
    x1: float,
    y1: float,
    cx: float,
    cy: float,
    ccw: bool,
    max_seg_deg: float = 5.0,
) -> List[tuple[float, float]]:
    """
```

Aproxima um arco circular (G02/G03) por uma sequência de pontos.

- ccw = True → arco anti-horário (G03)
- ccw = False → arco horário (G02)

Retorna pontos **sem** o ponto inicial (x0,y0), e **com** o ponto final (x1,y1). O chamador deve garantir que já possui (x0,y0).

```
    """
    r0 = math.hypot(x0 - cx, y0 - cy)
    r1 = math.hypot(x1 - cx, y1 - cy)
    r = (r0 + r1) / 2.0 if (r0 > 0 and r1 > 0) else max(r0, r1)
    if r == 0:
        return [(x1, y1)]

    a0 = math.atan2(y0 - cy, x0 - cx)
    a1 = math.atan2(y1 - cy, x1 - cx)
    if ccw:
        if a1 <= a0:
            a1 += 2.0 * math.pi
    else:
        if a1 >= a0:
            a1 -= 2.0 * math.pi

    sweep = a1 - a0
    max_seg_rad = max_seg_deg * math.pi / 180.0
    n_seg = max(4, int(math.ceil(abs(sweep) / max_seg_rad)))

    pts: List[tuple[float, float]] = []
    for k in range(1, n_seg + 1):
        t = a0 + sweep * (k / n_seg)
        x = cx + r * math.cos(t)
        y = cy + r * math.sin(t)
        pts.append((x, y))
    return pts
```

```
def _build_layer_core_mm(
    gerber_lines: List[str],
```

```

macros: Dict[str, ApertureMacro],
apertures: Dict[int, ApertureInstance],
cfg: GerberConfig,
) -> Tuple[List[List[Point]], List[GerberObject]]:
    """

```

Núcleo comum:

- interpreta o Gerber e gera
(lista de polígonos, lista de GerberObject).

"""

"""

Percorre o arquivo Gerber inteiro e retorna uma lista de polígonos em coordenadas absolutas ****em milímetros****, combinando:

- flashes D03 (pads instanciados via aberturas e macros)
- regiões G36/G37 (polígonos sólidos)

"""

```
all_polys: List[List[Point]] = []
```

```
all_objects: List[GerberObject] = []
```

```
current_dcode: int | None = None
```

```
last_draw_mode: str | None = None # "D01", "D02", "D03"
```

```
current_g: int | None = None # último G de interpolação (1,2,3)
```

```
cur_x_mm: float | None = None
```

```
cur_y_mm: float | None = None
```

```
region_active = False
```

```
region_pts: List[tuple[float, float]] = []
```

```
def coord_to_mm(v_base: float) -> float:
```

```
    return to_mm_from_unit(v_base, cfg)
```

```
# Converte texto em registros separados por '*'
```

```
records = gerber_to_records(gerber_lines)
```

```
for rec in records:
```

```
    line = rec.strip()
```

```
    if not line:
```

```
        continue
```

```
# Comentários e comandos de configuração são ignorados aqui
```

```
if line.startswith("G04") or line.startswith("%"):
```

```
    continue
```

```
# Atualiza G-code atual (modo de interpolação) quando aparece
```

```
g_tmp = _get_g_code(line)
```

```
if g_tmp in (1, 2, 3): # G01, G02, G03
```

```
    current_g = g_tmp
```

```
# Início/fim de região (G36/G37)
```

```
if "G36*" in line:
```

```

region_active = True
region_pts = []

continue
if "G37*" in line:
    if region_active and len(region_pts) > 1:
        if region_pts[0] != region_pts[-1]:
            region_pts.append(region_pts[0])
            poly = region_pts.copy()

            all_polys.append(poly)

        # Cria objeto de região
        try:
            xs = [p[0] for p in poly]
            ys = [p[1] for p in poly]
            cx = (min(xs) + max(xs)) / 2.0
            cy = (min(ys) + max(ys)) / 2.0
        except Exception:
            cx = cy = None

        obj = GerberObject(
            id=len(all_objects),
            kind="region",
            dcode=None,
            x_mm=cx,
            y_mm=cy,
            params={},
            polygon_mm=poly,
        )
        all_objects.append(obj)
    region_active = False
    region_pts = []
    continue

# Seleção de D-code: G54Dnn* ou Dnn* sozinho (sem X/Y)
if line.startswith("G54D"):
    idx = line.index("D") + 1
    j = idx
    while j < len(line) and line[j].isdigit():
        j += 1
    if j > idx:
        current_dcode = int(line[idx:j])
        continue
if (
    line.startswith("D")
    and "X" not in line
    and "Y" not in line
):
    idx = 1
    j = idx

```

```

while j < len(line) and line[j].isdigit():
    j += 1
if j > idx:
    current_dcode = int(line[idx:j])
    continue

# Determina o modo de desenho atual (D01/D02/D03), aceitando D1/D2/D3
draw_mode = _get_draw_mode(line)
if draw_mode is not None:
    last_draw_mode = draw_mode
else:
    draw_mode = last_draw_mode

# Guarda posição anterior antes de atualizar
prev_x_mm, prev_y_mm = cur_x_mm, cur_y_mm

# Atualiza coordenadas X/Y (em mm)
x_str, y_str = _parse_xy_from_line(line)
if x_str is not None:
    base_x = parse_coord(x_str, cfg)
    cur_x_mm = coord_to_mm(base_x)
if y_str is not None:
    base_y = parse_coord(y_str, cfg)
    cur_y_mm = coord_to_mm(base_y)

# Se não temos ainda coordenadas válidas, não há o que fazer
if cur_x_mm is None or cur_y_mm is None:
    continue

# Tratamento de regiões (G36/G37)
if region_active:
    g_code = current_g
    is_arc = g_code in (2, 3) # G02 (CW) ou G03 (CCW)

if draw_mode == "D02":
    region_pts = [(cur_x_mm, cur_y_mm)]
elif draw_mode == "D01":
    if not region_pts:
        region_pts = [(cur_x_mm, cur_y_mm)]
    else:
        if is_arc:
            start_x, start_y = region_pts[-1]
            i_str, j_str = _parse_ij_from_record(line)
            if i_str is not None or j_str is not None:
                i_base = parse_coord(i_str, cfg) if i_str else 0.0
                j_base = parse_coord(j_str, cfg) if j_str else 0.0
                i_mm = coord_to_mm(i_base)
                j_mm = coord_to_mm(j_base)
                cx = start_x + i_mm
                cy = start_y + j_mm
                ccw = (g_code == 3)

```

```

        arc_pts = _approx_arc_points(
            start_x, start_y,
            cur_x_mm, cur_y_mm,
            cx, cy,
            ccw=ccw,
        )
        region_pts.extend(arc_pts)
    else:
        region_pts.append((cur_x_mm, cur_y_mm))
    else:
        region_pts.append((cur_x_mm, cur_y_mm))
    continue

```

Flashes D03 (pads / furos do stencil)

if draw_mode == "D03" and current_dcode is not None:

```

    ap = apertures.get(current_dcode)

```

if ap is None:

```

    continue

```

if ap.kind == "circle":

```

    dia_mm = float(ap.params["dia_mm"])

```

```

    polys = circle_to_polys_mm(cur_x_mm, cur_y_mm, dia_mm)

```

for poly in polys:

```

    all_polys.append(poly)

```

```

    obj = GerberObject(

```

```

        id=len(all_objects),

```

```

        kind="flash_circle",

```

```

        dcode=current_dcode,

```

```

        x_mm=cur_x_mm,

```

```

        y_mm=cur_y_mm,

```

```

        params={"dia_mm": dia_mm},

```

```

        polygon_mm=poly,

```

```

    )

```

```

    all_objects.append(obj)

```

elif ap.kind == "rect":

```

    w_mm = float(ap.params["width_mm"])

```

```

    h_mm = float(ap.params["height_mm"])

```

```

    polys = rect_to_polys_mm(cur_x_mm, cur_y_mm, w_mm, h_mm)

```

for poly in polys:

```

    all_polys.append(poly)

```

```

    obj = GerberObject(

```

```

        id=len(all_objects),

```

```

        kind="flash_rect",

```

```

        dcode=current_dcode,

```

```

        x_mm=cur_x_mm,

```

```

        y_mm=cur_y_mm,

```

```

        params={"width_mm": w_mm, "height_mm": h_mm},

```

```

        polygon_mm=poly,

```

```

    )

```

```

    all_objects.append(obj)

```

elif ap.kind == "oval":

```

w_mm = float(ap.params["width_mm"])
h_mm = float(ap.params["height_mm"])
polys = oval_to_polys_mm(cur_x_mm, cur_y_mm, w_mm, h_mm)
for poly in polys:
    all_polys.append(poly)
    obj = GerberObject(
        id=len(all_objects),
        kind="flash_oval",
        dcode=current_dcode,
        x_mm=cur_x_mm,
        y_mm=cur_y_mm,
        params={"width_mm": w_mm, "height_mm": h_mm},
        polygon_mm=poly,
    )
    all_objects.append(obj)
elif ap.kind == "macro":
    macro_name = ap.params.get("macro_name")
    mac = macros.get(macro_name)
    if mac is None:
        continue
    polys = mac.render(
        scale_x=1.0,
        scale_y=1.0,
        rot_deg=0.0,
        trans=(cur_x_mm, cur_y_mm),
    )
    for poly in polys:
        all_polys.append(poly)
        obj = GerberObject(
            id=len(all_objects),
            kind="flash_macro",
            dcode=current_dcode,
            x_mm=cur_x_mm,
            y_mm=cur_y_mm,
            params={"macro_name": macro_name or ""},
            polygon_mm=poly,
        )
        all_objects.append(obj)

return all_polys, all_objects

```

```

def build_layer_polys_mm(
    gerber_lines: List[str],
    macros: Dict[str, ApertureMacro],
    apertures: Dict[int, ApertureInstance],
    cfg: GerberConfig,
) -> List[List[Point]]:
    """

```

Interface antiga preservada:

- retorna apenas a lista de polígonos em mm.

```

"""
polys, _ = _build_layer_core_mm(gerber_lines, macros, apertures, cfg)
return polys

def build_layer_objects_mm(
    gerber_lines: List[str],
    macros: Dict[str, ApertureMacro],
    apertures: Dict[int, ApertureInstance],
    cfg: GerberConfig,
) -> List[GerberObject]:
    """
    Nova interface:
    - retorna a lista de objetos Gerber (flash/região) já em mm,
      com tipo, D-code, posição, parâmetros e polígono associado.
    """
    _, objects = _build_layer_core_mm(gerber_lines, macros, apertures, cfg)
    return objects

```

File: render.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/testes_gerber/gerber_viewer

```

from __future__ import annotations

from typing import List, Tuple

from PIL import Image, ImageDraw

def render_polys_to_image(
    polys: List[List[Tuple[float, float]]],
    img_size: tuple[int, int] = (500, 500),
    margin: int = 20,
    fill="black",
    bg="white",
) -> Image.Image:
    """
    Constrói um objeto PIL.Image a partir da lista de polígonos.
    """
    # Extraí todos os X e Y para determinar bounds
    xs = [pt[0] for poly in polys for pt in poly]
    ys = [pt[1] for poly in polys for pt in poly]
    minx, maxx = min(xs), max(xs)
    miny, maxy = min(ys), max(ys)

    # Tamanho do "mundo"
    w_world = maxx - minx
    h_world = maxy - miny
    if w_world == 0 or h_world == 0:

```

```

raise RuntimeError("Polígonos degenerados")

# Fator de escala para caber na imagem (mantendo aspecto)
scale_x = (img_size[0] - 2 * margin) / w_world
scale_y = (img_size[1] - 2 * margin) / h_world
scale = min(scale_x, scale_y)

# Cria imagem
img = Image.new("RGB", img_size, bg)
draw = ImageDraw.Draw(img)

# Desenha cada polígono, invertendo Y para o sistema de imagem
for poly in polys:
    pts = []
    for x, y in poly:
        xx = (x - minx) * scale + margin
        yy = img_size[1] - ((y - miny) * scale + margin)
        pts.append((xx, yy))
    draw.polygon(pts, fill=fill)

return img

def draw_polys(
    polys: List[List[tuple[float, float]]],
    filename: str = "macro.png",
    img_size: tuple[int, int] = (500, 500),
    margin: int = 20,
) -> None:
    """
    Desenha a lista de polígonos em um PNG.
    """
    img = render_polys_to_image(polys, img_size=img_size, margin=margin)
    img.save(filename)
    print(f"Imagem salva em: {filename}")

```

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/testes_gerber/gerber_viewer

```

from .constants import INCH_TO_MM
from .config import GerberConfig, parse_gerber_config, parse_coord
from .apertures import (
    ApertureMacro,
    ApertureInstance,
    parse_macro,
    parse_all_macros,
    parse_add,
)
from .geometry import (

```

```

        circle_to_polys_mm,
        rect_to_polys_mm,
        oval_to_polys_mm,
    )
    from .parser import build_layer_polys_mm, build_layer_objects_mm, GerberObject
    from .exporter import objects_to_gerber
    from .render import render_polys_to_image, draw_polys

__all__ = [
    "INCH_TO_MM",
    "GerberConfig",
    "parse_gerber_config",
    "parse_coord",
    "ApertureMacro",
    "ApertureInstance",
    "parse_macro",
    "parse_all_macros",
    "parse_add",
    "circle_to_polys_mm",
    "rect_to_polys_mm",
    "oval_to_polys_mm",
    "build_layer_polys_mm",
    "build_layer_objects_mm",
    "GerberObject",
    "objects_to_gerber",
    "render_polys_to_image",
    "draw_polys",
]

```

File: mainwindow.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/testes_gerber/gerber_viewer/gui

```

from __future__ import annotations

import sys
import traceback
from typing import Dict, List

from PyQt6.QtCore import Qt
from PyQt6.QtGui import QColor, QAction
from PyQt6.QtWidgets import (
    QApplication,
    QColorDialog,
    QFileDialog,
    QHBoxLayout,
    InputDialog,
    QLabel,
    QMainWindow,
    QMenu,

```

```
QMessageBox,  
QPushButton,  
QSizePolicy,  
QVBoxLayout,  
QWidget,  
)
```

```
from ..geometry import circle_to_polys_mm, rect_to_polys_mm, oval_to_polys_mm  
from ..apertures import ApertureInstance, parse_add, parse_all_macros  
from ..config import GerberConfig, parse_gerber_config  
from ..parser import build_layer_objects_mm, GerberObject  
from ..exporter import objects_to_gerber  
from ..render import render_polys_to_image  
from .preview import PreviewGraphicsView
```

```
class GerberMacroViewer(QMainWindow):
```

```
    """
```

```
    Janela principal da aplicação (PyQt6):
```

- Menu para criar projeto e importar arquivo Gerber.
- Renderização da camada completa em mm logo após a importação.
- Área de preview com zoom/pan para visualizar a camada completa.

```
    """
```

```
def __init__(self):
```

```
    super().__init__()
```

```
    self.setWindowTitle("Gerber Macro Viewer")
```

```
    # Dimensões da imagem base (alta resolução para zoom)
```

```
    self.preview_width = 2000
```

```
    self.preview_height = 2000
```

```
    # Armazena dados do Gerber carregado
```

```
    self.gerber_lines: List[str] | None = None
```

```
    self.gerber_cfg: GerberConfig | None = None
```

```
    # Armazena macros carregadas (%AM...) e aperturas (%ADD...)
```

```
    self.macros = {}
```

```
    self.apertures_by_dcode: Dict[int, ApertureInstance] = {}
```

```
    # Cores configuráveis pelo usuário
```

```
    self.aperture_color = QColor(Qt.GlobalColor.black)
```

```
    self.background_color = QColor(Qt.GlobalColor.white)
```

```
    # Guarda polígonos da camada para exportar (PNG) e objetos para edição
```

```
    self._full_layer_polys_mm: List[List[tuple[float, float]]] | None = None
```

```
    self._full_layer_objects: List[GerberObject] | None = None
```

```
    self._current_pixmap = None # mantido apenas para compatibilidade
```

```
    self.current_project_name: str | None = None
```

```
    self._build_ui()
```

```

# ----- UI
def _build_ui(self):
    menubar = self.menuBar()

    # ----- Menu Arquivo -----
    arquivo_menu: QMenu = menubar.addMenu("Arquivo")

    # Novo projeto
    act_new_project: QAction = arquivo_menu.addAction("Novo projeto")
    act_new_project.triggered.connect(self.on_new_project)

    # Importar arquivo .gbr
    act_import_gbr: QAction = arquivo_menu.addAction("Importar arquivo .gbr")
    act_import_gbr.triggered.connect(self.on_import_gbr)

    arquivo_menu.addSeparator()

    # Salvar / Salvar como
    act_save: QAction = arquivo_menu.addAction("Salvar")
    act_save.triggered.connect(self.on_save)
    act_save_as: QAction = arquivo_menu.addAction("Salvar como")
    act_save_as.triggered.connect(self.on_save_as)

    arquivo_menu.addSeparator()

    # Exportar
    export_menu: QMenu = arquivo_menu.addMenu("Exportar")
    act_export_gbr: QAction = export_menu.addAction(".gbr")
    act_export_gbr.triggered.connect(self.on_export_gbr)
    act_export_dxf: QAction = export_menu.addAction(".dxf")
    act_export_dxf.triggered.connect(self.on_export_dxf)

    # ----- Menu Configurações -----
    config_menu = menubar.addMenu("Configurações")

    act_aperture_color = config_menu.addAction("Cor da abertura")
    act_aperture_color.triggered.connect(self.on_choose_aperture_color)

    act_bg_color = config_menu.addAction("Cor do fundo")
    act_bg_color.triggered.connect(self.on_choose_background_color)

    central = QWidget(self)
    self.setCentralWidget(central)
    main_layout = QVBoxLayout(central)

    # Barra superior
    top_layout = QHBoxLayout()

    self.reset_view_btn = QPushButton("Ajustar visão")
    self.reset_view_btn.clicked.connect(self.on_reset_view)
    self.reset_view_btn.setEnabled(False)

```

```

top_layout.addWidget(self.reset_view_btn)

self.export_btn = QPushButton("Exportar PNG...")
self.export_btn.clicked.connect(self.on_export_png)
self.export_btn.setEnabled(False)
top_layout.addWidget(self.export_btn)

top_layout.addStretch(1)
main_layout.addLayout(top_layout)

# Área de preview
right_layout = QVBoxLayout()
main_layout.addLayout(right_layout, 1)

lbl_preview = QLabel("Pré-visualização da camada completa:")
right_layout.addWidget(lbl_preview)

self.preview_view = PreviewGraphicsView()
self.preview_view.setSizePolicy(
    QSizePolicy.Policy.Expanding,
    QSizePolicy.Policy.Expanding,
)
self.preview_view.setMinimumSize(600, 400)
# Conecta sinal de exclusão de objeto vindo do preview
self.preview_view.objectDeleteRequested.connect(self.on_delete_object)
self.preview_view.objectEditRequested.connect(self.on_edit_object)
right_layout.addWidget(self.preview_view, 1)
# Barra de status (rodapé) para exibir o nome do projeto
self.statusBar().showMessage("Nenhum projeto")

# ----- Menu Arquivo
def _feature_not_implemented(self, name: str):
    QMessageBox.information(
        self,
        "Não implementado",
        f"A funcionalidade '{name}' ainda não foi implementada.",
    )

def on_new_project(self):
    """
    Cria um novo projeto:
    - limpa a visualização/estado atual;
    - solicita o nome do projeto (obrigatório);
    - exibe o nome no rodapé no formato 'Projeto: <nome>'.
    """
    # 1) Limpa visualização e estado atual de arquivo/projeto
    self.gerber_lines = None
    self.gerber_cfg = None
    self.apertures_by_dcode = {}
    self.macros = {}
    self._full_layer_polys_mm = None

```

```
self._full_layer_objects = None
self._clear_preview()
```

```
# 2) Solicita o nome do novo projeto (obrigatório)
```

```
name, ok = QDialog.getText(
    self,
    "Novo projeto",
    "Informe o nome do novo projeto:",
)
```

```
if not ok:
```

```
    # Usuário cancelou: não altera o projeto atual
    return
```

```
name = name.strip()
```

```
if not name:
```

```
    QMessageBox.warning(self, "Nome obrigatório",
        "É necessário informar um nome para o projeto.")
    return
```

```
# 3) Atualiza estado e barra de status
```

```
self.current_project_name = name
```

```
self.statusBar().showMessage(f"Projeto: {name}")
```

```
# 4) Inicia fluxo de importação de arquivo Gerber para este projeto
```

```
# (abre o diálogo de seleção e, se o arquivo for carregado,
```

```
# já renderiza automaticamente a camada completa)
```

```
self.on_import_gbr()
```

```
def on_import_gbr(self):
```

```
    """
```

```
    Importa um arquivo Gerber (.gbr):
```

- reutiliza a rotina de abertura de arquivo;
- se houver arquivo selecionado e carregado com sucesso,
já gera e exibe a camada completa (mm).

```
    """
```

```
loaded = self.on_open_file()
```

```
if not loaded:
```

```
    return
```

```
self.on_render_full_layer()
```

```
def on_save(self):
```

```
    self._feature_not_implemented("Salvar")
```

```
def on_save_as(self):
```

```
    self._feature_not_implemented("Salvar como")
```

```
def on_export_gbr(self):
```

```
    """
```

```
    Exporta a camada atual (objetos em mm) para um novo arquivo Gerber,  
    em um formato RS-274X simplificado:
```

- Unidade: milímetros (%MOMM*%)
- Formato: FSLAX33Y33 (3 inteiros, 3 decimais)
- Todos os objetos são convertidos em regiões sólidas (G36/G37).

"""

```
if not self._full_layer_objects:
    QMessageBox.information(
        self,
        "Nada para exportar",
        "Não há nenhum objeto carregado/gerado para exportar.\n"
        "Crie um projeto, importe um Gerber e gere a camada completa.",
    )
    return
```

```
path, _ = QFileDialog.getSaveFileName(
    self,
    "Exportar como Gerber",
    "",
    "Arquivos Gerber (*.gbr *.ger *.pho *.art *.gb* *.gt* *.g*)",
)
```

```
if not path:
    return
```

```
try:
    lines = objects_to_gerber(self._full_layer_objects)
    with open(path, "w", encoding="utf-8") as f:
        f.writelines(lines)
except Exception:
    traceback.print_exc()
    QMessageBox.critical(
        self,
        "Erro ao exportar Gerber",
        "Ocorreu um erro ao gerar o arquivo Gerber exportado.\n"
        "Veja o terminal para detalhes.",
    )
```

```
def on_export_dxf(self):
    self._feature_not_implemented("Exportar .dxf")
```

----- Abrir arquivo (.gbr)

(rotina de abertura/reimportação de arquivo Gerber)

```
def on_open_file(self) -> bool:
```

```
    filters = (
        "Arquivos Gerber (*.gbr *.ger *.pho *.art *.gb* *.gt* *.g*);;"
        "Todos os arquivos (*.*)"
    )
```

```
    path, _ = QFileDialog.getOpenFileName(
        self,
        "Selecionar arquivo Gerber",
        "",
        filters,
    )
```

```
if not path:
    return False
```

```
self.gerber_lines = None
self.gerber_cfg = None
self.apertures_by_dcode = {}
self._full_layer_polys_mm = None
self._full_layer_objects = None
self.macros = {}
```

```
try:
    with open(path, "r", encoding="utf-8", errors="ignore") as f:
        lines = f.readlines()
except OSError as e:
    QMessageBox.critical(
        self,
        "Erro ao abrir arquivo",
        f"Não foi possível abrir:\n{path}\n\n{e}",
    )
    return False
```

```
self.gerber_lines = lines
self.gerber_cfg = parse_gerber_config(lines)
self.apertures_by_dcode = parse_add(lines, self.gerber_cfg)
self.macros = parse_all_macros(lines)
```

```
self._clear_preview()
return True
```

```
# ----- Camada completa
```

```
def on_render_full_layer(self):
    if self.gerber_lines is None or self.gerber_cfg is None:
        QMessageBox.information(
            self,
            "Nenhum arquivo",
            "Abra um arquivo Gerber antes de gerar a camada completa.",
        )
    return
```

```
try:
    # Gera lista de objetos Gerber (flash/região) em mm
    objects = build_layer_objects_mm(
        self.gerber_lines,
        self.macros,
        self.apertures_by_dcode,
        self.gerber_cfg,
    )
    self._full_layer_objects = objects
```

```
# Deriva a lista de polígonos a partir dos objetos (para PNG, etc.)
polys_mm: List[List[tuple[float, float]]] = [
```

```

        obj.polygon_mm for obj in objects
        if obj.polygon_mm and len(obj.polygon_mm) >= 3
    ]
    self._full_layer_polys_mm = polys_mm
except Exception:
    traceback.print_exc()
    QMessageBox.critical(
        self,
        "Erro ao gerar camada completa",
        "Ocorreu um erro ao processar a camada completa.\n"
        "Veja o terminal para detalhes.",
    )
    return

if not polys_mm:
    QMessageBox.information(
        self,
        "Nada para exibir",
        "Nenhuma entidade geométrica foi encontrada para a camada completa.",
    )
    return

try:
    # Apenas para validar rasterização; preview será vetorial
    _ = render_polys_to_image(
        polys_mm,
        img_size=(self.preview_width, self.preview_height),
        margin=20,
    )
except Exception:
    traceback.print_exc()
    QMessageBox.critical(
        self,
        "Erro ao renderizar imagem",
        "Falha ao rasterizar a camada completa.",
    )
    return

try:
    self._current_pixmap = None
    # Usa a nova interface baseada em objetos, se disponível
    if self._full_layer_objects:
        self.preview_view.set_objects(
            self._full_layer_objects,
            aperture_color=self.aperture_color,
            bg_color=self.background_color,
        )
    else:
        # fallback (não deve ocorrer nesse fluxo normal)
        self.preview_view.set_polygons(
            polys_mm,

```

```

        aperture_color=self.aperture_color,
        bg_color=self.background_color,
    )
    self.export_btn.setEnabled(True)
    self.reset_view_btn.setEnabled(True)
except Exception:
    traceback.print_exc()
    QMessageBox.critical(
        self,
        "Erro ao converter imagem",
        "Não foi possível converter a imagem da camada completa para exibição.",
    )

```

----- Export PNG

```

def on_export_png(self):
    if not self._full_layer_polys_mm:
        QMessageBox.information(
            self,
            "Nada para exportar",
            "Não há nenhum dado de camada completa para exportar.\n"
            "Gere a camada completa primeiro.",
        )
    return

```

```

path, _ = QFileDialog.getSaveFileName(
    self,
    "Salvar imagem em PNG",
    "",
    "Imagem PNG (*.png)",
)
if not path:
    return

```

```

factor, ok = QInputDialog.getInt(
    self,
    "Escala da imagem",
    "Fator de escala (1 = igual ao preview, 2 = 2x, ...):",
    4, # padrão
    1, # mínimo
    20, # máximo
    1, # step
)
if not ok:
    return

```

from ..render import render_polys_to_image # local import para evitar dependências cíclicas

```

try:
    ac = self.aperture_color
    bc = self.background_color
    fill_rgb = (ac.red(), ac.green(), ac.blue())

```

```
bg_rgb = (bc.red(), bc.green(), bc.blue())
```

```
img = render_polys_to_image(
    self._full_layer_polys_mm,
    img_size=(
        self.preview_width * factor,
        self.preview_height * factor,
    ),
    margin=20 * factor,
    fill=fill_rgb,
    bg=bg_rgb,
)
img.save(path, format="PNG")
except Exception:
    traceback.print_exc()
    QMessageBox.critical(
        self,
        "Erro ao exportar PNG",
        "Ocorreu um erro ao gerar o PNG de alta resolução.\n"
        "Veja o terminal para detalhes.",
    )
```

```
def on_reset_view(self):
    self.preview_view.reset_view()
```

```
# ----- Cores
```

```
def on_choose_aperture_color(self):
    color = QColorDialog.getColor(
        self.aperture_color,
        self,
        "Selecionar cor da abertura",
    )
    if not color.isValid():
        return
    self.aperture_color = color
    # Re-renderiza baseado em objetos se possível (mantém mapeamento)
    if self._full_layer_objects:
        self.preview_view.set_objects(
            self._full_layer_objects,
            aperture_color=self.aperture_color,
            bg_color=self.background_color,
        )
    elif self._full_layer_polys_mm:
        self.preview_view.set_polygons(
            self._full_layer_polys_mm,
            aperture_color=self.aperture_color,
            bg_color=self.background_color,
        )
```

```
# ----- Edição: alterar propriedades do objeto
```

```
def on_edit_object(self, index: int):
```

''''''

Edita propriedades geométricas de um objeto simples:

- flash_circle -> diâmetro
- flash_rect -> largura x altura
- flash_oval -> largura x altura

Regiões (kind="region") também podem ser redimensionadas (largura x altura) por escala do polígono.

''''''

```
if self._full_layer_objects is None:
    print("[DEBUG edit] _full_layer_objects is None")
    return
if not (0 <= index < len(self._full_layer_objects)):
    print(
        f"[DEBUG edit] index fora do intervalo: "
        f"idx={index}, n={len(self._full_layer_objects)}"
    )
    return

obj = self._full_layer_objects[index]

print(
    "[DEBUG edit] objeto selecionado:",
    f"idx={index}, kind={obj.kind}, dcode={obj.dcode}, "
    f"x={obj.x_mm}, y={obj.y_mm}, params={obj.params}",
)

# Diálogo específico por tipo
if obj.kind == "flash_circle":
    cur_dia = float(obj.params.get("dia_mm", 0.0))
    new_dia, ok = QInputDialog.getDouble(
        self,
        "Editar diâmetro",
        "Novo diâmetro (mm):",
        cur_dia,
        0.001,
        1000.0,
        3, # casas decimais
    )
    if not ok:
        return
    if new_dia <= 0:
        QMessageBox.warning(self, "Valor inválido",
            "O diâmetro deve ser maior que zero.")
        return

    obj.params["dia_mm"] = new_dia

# Recalcula o polígono do círculo
if obj.x_mm is None or obj.y_mm is None:
    return
```

```

polys = circle_to_polys_mm(obj.x_mm, obj.y_mm, new_dia)
obj.polygon_mm = polys[0]

elif obj.kind in ("flash_rect", "flash_oval"):
    cur_w = float(obj.params.get("width_mm", 0.0))
    cur_h = float(obj.params.get("height_mm", 0.0))

    # Pergunta largura
    new_w, ok = QInputDialog.getDouble(
        self,
        "Editar largura",
        "Nova largura (mm):",
        cur_w,
        0.001,
        1000.0,
        3,
    )
    if not ok:
        return

    # Pergunta altura
    new_h, ok = QInputDialog.getDouble(
        self,
        "Editar altura",
        "Nova altura (mm):",
        cur_h,
        0.001,
        1000.0,
        3,
    )
    if not ok:
        return

    if new_w <= 0 or new_h <= 0:
        QMessageBox.warning(self, "Valores inválidos",
            "Largura e altura devem ser maiores que zero.")
        return

    obj.params["width_mm"] = new_w
    obj.params["height_mm"] = new_h

    if obj.x_mm is None or obj.y_mm is None:
        return
    if obj.kind == "flash_rect":
        polys = rect_to_polys_mm(obj.x_mm, obj.y_mm, new_w, new_h)
    else: # flash_oval
        polys = oval_to_polys_mm(obj.x_mm, obj.y_mm, new_w, new_h)
    obj.polygon_mm = polys[0]
elif obj.kind == "region":
    # Edição genérica de região: redimensiona largura x altura
    if not obj.polygon_mm or len(obj.polygon_mm) < 3:

```

```

    QMessageBox.information(
        self,
        "Não editável",
        "Esta região não possui polígono válido para edição.",
    )
    return

xs = [p[0] for p in obj.polygon_mm]
ys = [p[1] for p in obj.polygon_mm]
minx, maxx = min(xs), max(xs)
miny, maxy = min(ys), max(ys)
cur_w = maxx - minx
cur_h = maxy - miny

if cur_w <= 0 or cur_h <= 0:
    QMessageBox.information(
        self,
        "Não editável",
        "Não foi possível determinar largura/altura da região.",
    )
    return

# Centro geométrico aproximado (igual ao usado no parser)
cx = (minx + maxx) / 2.0
cy = (miny + maxy) / 2.0

# Pergunta nova largura
new_w, ok = QInputDialog.getDouble(
    self,
    "Editar largura da região",
    "Nova largura (mm):",
    cur_w,
    0.001,
    1000.0,
    3,
)
if not ok:
    return

# Pergunta nova altura
new_h, ok = QInputDialog.getDouble(
    self,
    "Editar altura da região",
    "Nova altura (mm):",
    cur_h,
    0.001,
    1000.0,
    3,
)
if not ok:
    return

```

```

if new_w <= 0 or new_h <= 0:
    QMessageBox.warning(
        self,
        "Valores inválidos",
        "Largura e altura devem ser maiores que zero.",
    )
    return

```

```

sx = new_w / cur_w
sy = new_h / cur_h

```

```

new_poly = []
for x, y in obj.polygon_mm:
    nx = cx + (x - cx) * sx
    ny = cy + (y - cy) * sy
    new_poly.append((nx, ny))

```

```

# Garante fechamento explícito
if new_poly and new_poly[0] != new_poly[-1]:
    new_poly.append(new_poly[0])

```

```

obj.polygon_mm = new_poly

```

```

else:
    # Por enquanto não editamos macros ou outros tipos
    QMessageBox.information(
        self,
        "Não editável",
        "Este tipo de objeto ainda não pode ser editado.",
    )
    return

```

```

# Atualiza lista de polígonos a partir dos objetos
self._full_layer_polys_mm = [
    o.polygon_mm
    for o in self._full_layer_objects
    if o.polygon_mm and len(o.polygon_mm) >= 3
]

```

```

# Re-renderiza
self.preview_view.set_objects(
    self._full_layer_objects,
    aperture_color=self.aperture_color,
    bg_color=self.background_color,
)

```

----- Edição: exclusão de objeto

```

def on_delete_object(self, index: int):
    """

```

Remove um objeto Gerber (flash/região) da camada atual e re-renderiza o preview.

Chamado quando o usuário escolhe "Excluir objeto" no menu de contexto da área de preview.

"""

```
if self._full_layer_objects is None:
    return
```

```
if not (0 <= index < len(self._full_layer_objects)):
    return
```

```
obj = self._full_layer_objects[index]
```

```
# Confirmação com o usuário
```

```
desc = obj.kind
```

```
if obj.dcode is not None:
```

```
    desc += f" (D{obj.dcode})"
```

```
msg = (
```

```
    "Tem certeza que deseja excluir este objeto?\n\n"
```

```
    f"Tipo: {obj.kind}\n"
```

```
)
```

```
if obj.dcode is not None:
```

```
    msg += f"D-code: D{obj.dcode}\n"
```

```
if obj.x_mm is not None and obj.y_mm is not None:
```

```
    msg += f"Posição: ({obj.x_mm:.6f}, {obj.y_mm:.6f}) mm\n"
```

```
reply = QMessageBox.question(
```

```
    self,
```

```
    "Confirmar exclusão",
```

```
    msg,
```

```
    QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
```

```
    QMessageBox.StandardButton.No,
```

```
)
```

```
if reply != QMessageBox.StandardButton.Yes:
```

```
    return
```

```
# Remove o objeto da lista
```

```
del self._full_layer_objects[index]
```

```
# Reconstrói a lista de polígonos
```

```
self._full_layer_polys_mm = [
```

```
    o.polygon_mm
```

```
    for o in self._full_layer_objects
```

```
    if o.polygon_mm and len(o.polygon_mm) >= 3
```

```
]
```

```
# Se não sobrou nada, limpa a tela
```

```
if not self._full_layer_objects:
```

```
    self._clear_preview()
```

```
    return
```

```
# Caso contrário, re-renderiza a partir da lista atualizada
```

```

self.preview_view.set_objects(
    self._full_layer_objects,
    aperture_color=self.aperture_color,
    bg_color=self.background_color,
)

```

```

def on_choose_background_color(self):
    color = QColorDialog.getColor(
        self.background_color,
        self,
        "Selecionar cor do fundo",
    )
    if not color.isValid():
        return
    self.background_color = color
    if self._full_layer_objects:
        self.preview_view.set_objects(
            self._full_layer_objects,
            aperture_color=self.aperture_color,
            bg_color=self.background_color,
        )
    elif self._full_layer_polys_mm:
        self.preview_view.set_polygons(
            self._full_layer_polys_mm,
            aperture_color=self.aperture_color,
            bg_color=self.background_color,
        )

```

----- Utilitários

```

def _clear_preview(self):
    self.preview_view._scene.clear()
    self.preview_view._polys_mm = []
    self.preview_view._objects = None
    self._current_pixmap = None
    self._full_layer_objects = None
    self.export_btn.setEnabled(False)
    self.reset_view_btn.setEnabled(False)

```

```

def run():
    app = QApplication(sys.argv)
    window = GerberMacroViewer()
    window.resize(900, 700)
    window.show()
    sys.exit(app.exec())

```

File: preview.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/testes_gerber/gerber_viewer/gui

```
from __future__ import annotations
```

```
import traceback
```

```
from typing import List, Optional
```

```
from PyQt6.QtCore import Qt, QRectF, pyqtSignal
```

```
from PyQt6.QtGui import QBrush, QColor, QPainter, QPainterPath, QPen
```

```
from PyQt6.QtWidgets import (
```

```
    QGraphicsItem,
```

```
    QGraphicsScene,
```

```
    QGraphicsView,
```

```
    QMenu,
```

```
    QMessageBox,
```

```
)
```

```
from ..parser import GerberObject
```

```
class PreviewGraphicsView(QGraphicsView):
```

```
    """
```

```
    Área de preview com suporte a:
```

- zoom com scroll do mouse;
- pan (arrastar) com botão do meio (scroll) pressionado;
- reset da visão (ajustar à área disponível).

```
    Renderiza os polígonos diretamente como vetores (QPainterPath).
```

```
    """
```

```
# Emite o índice do objeto/polígono para exclusão ou edição
```

```
objectDeleteRequested = pyqtSignal(int)
```

```
objectEditRequested = pyqtSignal(int)
```

```
def __init__(self, parent=None):
```

```
    super().__init__(parent)
```

```
    self._scene = QGraphicsScene(self)
```

```
    self.setScene(self._scene)
```

```
    self._zoom = 1.0
```

```
# Guarda os polígonos em mm na mesma ordem em que foram desenhados
```

```
self._polys_mm: List[List[tuple[float, float]]] = []
```

```
# Opcional: lista de objetos Gerber associados a esses polígonos
```

```
self._objects: Optional[List[GerberObject]] = None
```

```
# Conjunto de índices atualmente selecionados (para possível uso futuro)
```

```
self._selected_indices: set[int] = set()
```

```
self.setRenderHints(
```

```
    QPainter.RenderHint.Antialiasing
```

```
    | QPainter.RenderHint.SmoothPixmapTransform
```

```
)
```

```
self.setTransformationAnchor(
```

```

        QGraphicsView.ViewportAnchor.AnchorUnderMouse
    )
    self.setResizeAnchor(
        QGraphicsView.ViewportAnchor.AnchorViewCenter
    )

    # Para receber eventos de teclado (Delete etc.)
    self.setFocusPolicy(Qt.FocusPolicy.StrongFocus)

    # Controle de pan
    self._panning = False
    self._last_mouse_pos = None

def reset_view(self):
    """
    Restaura o zoom/pan para enquadrar toda a imagem
    na área visível, mantendo o aspecto.
    """
    self.resetTransform()
    self._zoom = 1.0
    if not self._scene.items():
        return
    self.fitInView(
        self._scene.sceneRect(),
        Qt.AspectRatioMode.KeepAspectRatio,
    )

def set_objects(
    self,
    objects: List[GerberObject],
    aperture_color: QColor,
    bg_color: QColor,
):
    """
    Nova interface: recebe a lista de GerberObject e desenha seus
    polígonos (polygon_mm). A ordem dos objetos é usada como índice
    para mapear itens gráficos → objeto.

    Por compatibilidade, internamente chamamos set_polygons().
    """
    # Corrige o mapeamento índice do polígono -> objeto:
    # mantém apenas objetos com polígono válido, na mesma ordem.
    self._objects = []
    polys_mm: List[List[tuple[float, float]]] = []

    for obj in objects or []:
        poly = obj.polygon_mm
        if not poly or len(poly) < 3:
            continue
        self._objects.append(obj)
        polys_mm.append(poly)

```

```
# Limpa seleção ao recarregar
```

```
self._selected_indices.clear()
```

```
self.set_polygons(polys_mm, aperture_color, bg_color)
```

```
def set_polygons(
```

```
    self,
```

```
    polys_mm: List[List[tuple[float, float]]],
```

```
    aperture_color: QColor,
```

```
    bg_color: QColor,
```

```
):
```

```
    """
```

Recebe a lista de polígonos em coordenadas de mundo (mm) e desenha
vetorialmente na cena (QGraphicsScene) usando QPainterPath.

Também armazena esses polígonos para permitir inspeção por clique.

```
    """
```

```
self._scene.clear()
```

```
self._zoom = 1.0
```

```
self.resetTransform()
```

```
self._polys_mm = polys_mm or []
```

```
# Limpa seleção atual
```

```
self._selected_indices.clear()
```

```
# Se estiver apenas usando polígonos, limpa os objetos associados
```

```
if not polys_mm:
```

```
    self._objects = None
```

```
if not polys_mm:
```

```
    return
```

```
# cor de fundo
```

```
self._scene.setBackgroundBrush(bg_color)
```

```
# Determina bounding box em mm
```

```
xs = [pt[0] for poly in polys_mm for pt in poly]
```

```
ys = [pt[1] for poly in polys_mm for pt in poly]
```

```
minx, maxx = min(xs), max(xs)
```

```
miny, maxy = min(ys), max(ys)
```

```
w = maxx - minx
```

```
h = maxy - miny
```

```
if w == 0 or h == 0:
```

```
    return
```

```
brush = QBrush(aperture_color)
```

```
pen = QPen(Qt.PenStyle.NoPen)
```

```
for idx, poly in enumerate(polys_mm):
```

```
    if len(poly) < 3:
```

```
        continue
```

```
    path = QPainterPath()
```

```
    x0, y0 = poly[0]
```

```
    path.moveTo(x0, -y0)
```

```

for x, y in poly[1:]:
    path.lineTo(x, -y)
item = self._scene.addPath(path, pen, brush)
try:
    item.setData(0, idx)
    # Permite seleção por clique esquerdo (Qt gerencia visual)
    item.setFlag(QGraphicsItem.GraphicsItemFlag.ItemIsSelectable, True)
except Exception:
    traceback.print_exc()
item.setBrush(brush)

```

```

rect = QRectF(minx, -maxy, w, h)
self._scene.setSceneRect(rect)
self.reset_view()

```

```

def wheelEvent(self, event):
    """Zoom com scroll do mouse."""
    if not self._scene.items():
        return

```

```

    angle = event.angleDelta().y()
    if angle == 0:
        return

```

```

    factor = 1.25 if angle > 0 else 0.8
    old_zoom = self._zoom
    self._zoom *= factor
    self._zoom = max(0.05, min(self._zoom, 100.0))
    factor = self._zoom / old_zoom
    self.scale(factor, factor)

```

```

def _show_polygon_info(self, poly_index: int):
    """
    Mostra em uma MessageBox informações sobre o polígono / objeto
    clicado. Se houver objetos Gerber associados, usa esses dados;
    caso contrário, usa apenas o polígono.
    """
    # Tenta primeiro usar a lista de objetos Gerber (se disponível)
    obj: Optional[GerberObject] = None
    if self._objects is not None and 0 <= poly_index < len(self._objects):
        obj = self._objects[poly_index]
        poly = obj.polygon_mm
    else:
        # Fallback: usa apenas a lista de polígonos
        if not (0 <= poly_index < len(self._polys_mm)):
            return
        poly = self._polys_mm[poly_index]

    if not poly:
        return

```

```

xs = [p[0] for p in poly]
ys = [p[1] for p in poly]
minx, maxx = min(xs), max(xs)
miny, maxy = min(ys), max(ys)
cx = (minx + maxx) / 2.0
cy = (miny + maxy) / 2.0

```

```

lines = [
    f"Índice do polígono: {poly_index}",
    f"Número de pontos: {len(poly)}",
    f"Limits X: ({minx:.6f}, {maxx:.6f}) mm",
    f"Limits Y: ({miny:.6f}, {maxy:.6f}) mm",
    f"Centro aproximado: ({cx:.6f}, {cy:.6f}) mm",
]

```

```

# Se tivermos um GerberObject, acrescenta informações semânticas
if obj is not None:
    lines.append("")
    lines.append(f"Tipo: {obj.kind}")
    if obj.dcode is not None:
        lines.append(f"D-code: D{obj.dcode}")
    if obj.x_mm is not None and obj.y_mm is not None:
        lines.append(f"Posição do flash: ({obj.x_mm:.6f}, {obj.y_mm:.6f}) mm")

    if obj.kind.startswith("flash_"):
        # Alguns parâmetros típicos
        if "dia_mm" in obj.params:
            lines.append(f"Diâmetro: {float(obj.params['dia_mm']):.6f} mm")
        if "width_mm" in obj.params and "height_mm" in obj.params:
            lines.append(
                "Largura x Altura: "
                f"{float(obj.params['width_mm']):.6f} x "
                f"{float(obj.params['height_mm']):.6f} mm"
            )
        if "macro_name" in obj.params:
            lines.append(f"Macro: {obj.params['macro_name']}")

```

```

text = "\n".join(lines)
QMessageBox.information(self, "Informações do polígono", text)

```

```

def mousePressEvent(self, event):
    # Garante que a view receba eventos de teclado após clique
    self.setFocus()
    if event.button() == Qt.MouseButton.RightButton:
        scene_pos = self.mapToScene(event.pos())
        items = self._scene.items(scene_pos)
        if items:
            item = items[0]
            try:
                idx = int(item.data(0))
            except Exception:

```

```
    traceback.print_exc()
    event.accept()
    return
```

```
# Menu de contexto para o objeto clicado
menu = QMenu(self)
act_info = menu.addAction("Informações do objeto")
act_edit = menu.addAction("Editar propriedades...")
act_delete = menu.addAction("Excluir objeto")
```

```
global_pos = self.mapToGlobal(event.pos())
chosen = menu.exec(global_pos)
```

```
if chosen is act_info:
    try:
        self._show_polygon_info(idx)
    except Exception:
        traceback.print_exc()
elif chosen is act_edit:
    # Solicita ao "dono" que edite o objeto
    try:
        print(f"[DEBUG preview] objectEditRequested idx={idx}")
        self.objectEditRequested.emit(idx)
    except Exception:
        traceback.print_exc()
elif chosen is act_delete:
    # Solicita ao "dono" (janela principal) que exclua o objeto
    try:
        self.objectDeleteRequested.emit(idx)
    except Exception:
        traceback.print_exc()
```

```
event.accept()
return
```

```
else:
    # Clique direito em área vazia: nada a fazer por enquanto
    event.accept()
    return
```

```
# Pan agora é com botão do meio (scroll)
if event.button() == Qt.MouseButton.MiddleButton:
    self._panning = True
    self._last_mouse_pos = event.pos()
    self.setCursor(Qt.CursorShape.ClosedHandCursor)
    event.accept()
    return
```

```
# Clique esquerdo: deixa o QGraphicsView lidar com seleção
super().mousePressEvent(event)
```

```
def mouseMoveEvent(self, event):
```

```

if self._panning and self._last_mouse_pos is not None:
    delta = event.pos() - self._last_mouse_pos
    self._last_mouse_pos = event.pos()
    hbar = self.horizontalScrollBar()
    vbar = self.verticalScrollBar()
    hbar.setValue(hbar.value() - delta.x())
    vbar.setValue(vbar.value() - delta.y())
    event.accept()
else:
    super().mouseMoveEvent(event)

def mouseReleaseEvent(self, event):
    if event.button() == Qt.MouseButton.MiddleButton and self._panning:
        self._panning = False
        self.setCursor(Qt.CursorShape.ArrowCursor)
        event.accept()
    else:
        super().mouseReleaseEvent(event)

# ----- teclado (Delete)
def keyPressEvent(self, event):
    """
    Permite excluir objetos selecionados com a tecla Delete/Backspace.
    A confirmação é tratada pela janela principal (on_delete_object).
    """
    key = event.key()
    if key in (Qt.Key.Key_Delete, Qt.Key.Key_Backspace):
        items = self._scene.selectedItems()
        if not items:
            event.accept()
            return

        indices: set[int] = set()
        for it in items:
            try:
                idx = int(it.data(0))
                indices.add(idx)
            except Exception:
                traceback.print_exc()

        # Emite pedido de exclusão para cada índice selecionado
        for idx in sorted(indices):
            try:
                self.objectDeleteRequested.emit(idx)
            except Exception:
                traceback.print_exc()

        event.accept()
        return

# Teclas não tratadas: comportamento padrão

```

```
super().keyPressEvent(event)
```

File: __init__.py

Directory: C:/Users/sense/PycharmProjects/Tensiometro/testes_gerber/gerber_viewer/gui

```
from .preview import PreviewGraphicsView
```

```
from .mainwindow import GerberMacroViewer
```

```
__all__ = ["PreviewGraphicsView", "GerberMacroViewer"]
```